

Process

Martin Kellogg



Process

Today's agenda:

- **Reading Quiz**
- Development methodologies
- Planning, estimation, and risk
- Measuring progress

Announcements

- Individual Project Proposals **due Wednesday (9/16)**
 - one page, but it's an important page
- Fill out the **team preferences survey** by Friday
 - link on course calendar or “surveys” page on course website
 - or Discord (see my pinned message from Friday)

Process

Today's agenda:

- Reading Quiz
- **Development methodologies**
- Planning, estimation, and risk
- Measuring progress

Development methodologies

- Traditionally, a large component of undergrad Software Engineering classes

Development methodologies

- Traditionally, a large component of undergrad Software Engineering classes
- I'm not going to make you memorize the stages of the Waterfall method, or the tenets of Agile, or the like

Development methodologies

- Traditionally, a large component of undergrad Software Engineering classes
- I'm not going to make you memorize the stages of the Waterfall method, or the tenets of Agile, or the like
 - Why? No one actually follows these procedures to the letter

Development methodologies

- Traditionally, a large component of undergrad Software Engineering classes
- I'm not going to make you memorize the stages of the Waterfall method, or the tenets of Agile, or the like
 - Why? No one actually follows these procedures to the letter
- Instead, my goal in this lecture is to give you an overview of the traditional ways of organizing a software development effort and give you the vocabulary to talk about it

What is a process? A methodology?

Definition: a *software process* is the set of activities and associated results that produce a software product

What is a process? A methodology?

Definition: a *software process* is the set of activities and associated results that produce a software product

Definition: a *software development methodology* is a template for a process: a specific set of activities (usually accompanied by an animating philosophy) for a team to follow to achieve a desired result

What is a process? A methodology?

Definition: a *software process* is the set of activities and associated results that produce a software product

Definition: a *software development methodology* is a template for a process: a specific set of activities (usually accompanied by an animating philosophy) for a team to follow to achieve a desired result
e.g., the Agile manifesto

What is a process? A methodology?

Definition: a *software process* is the set of activities and associated results that produce a software product

Definition: a *software development methodology* is a template for a process: a specific set of activities (usually accompanied by an animating philosophy) for a team to follow to achieve a desired result

not a guarantee - just a goal

A list of methodologies

- Waterfall
- Spiral
- Agile
- Scrum
- Extreme Programming (XP)
- “wagile”

A list of methodologies

- Waterfall
- Spiral
- Agile
- Scrum

We'll discuss these four - you can look up the others on your own if you're curious

- Extreme Programming (XP)
- “wagile”

Why have a methodology at all?

Why have a methodology at all?

- Standardization among developers
- Shared language
- Estimation: your boss probably wants to know when you'll be able to ship!
- You implicitly have a process, whether you know it or not (and it might not be very good if you're not paying attention)

Why have a methodology at all?

- Standardization among developers
- Shared language
- Estimation: your boss probably wants to know when you'll be able to ship!
- You implicitly have a process, whether you know it or not (and it might not be very good if you're not paying attention)



sometimes this is called an *ad hoc* methodology

Examples of issues with ad hoc process

- **Requirements:** Mid-project informal agreement to changes suggested by customer or manager.

Examples of issues with ad hoc process

- **Requirements:** Mid-project info suggested by customer or management

Project scope expands (a lot!)

Examples of issues with ad hoc process

- **Requirements:** Mid-project informal agreement to changes suggested by customer or manager.
- **QA:** Late detection of requirements and design issues.
Test-debug-reimplement cycle limits development of new features.

Examples of issues with ad hoc process

- **Requirements:** Mid-project informal agreement to changes suggested by customer or manager.
- **QA:** Late detection of requirements
Test-debug-reimplement cycle

Release with known defects

Defect cost vs. detection time

- An IBM report gives an average defect repair cost of (2008\$):
 - \$25 during coding
 - \$100 at build time
 - \$450 during testing/QA
 - \$16,000 post-release

Examples of issues with ad hoc process

- **Requirements:** Mid-project informal agreement to changes suggested by customer or manager.
- **QA:** Late detection of requirements and design issues.
Test-debug-reimplement cycle limits development of new features.
- **Defect Tracking:** Bug reports collected informally.

Examples of issues with ad hoc process

- **Requirements:** Mid-project informal agreement to changes suggested by customer or manager.
- **QA:** Late detection of requirements and design issues.
Test-debug-reimplement cycle limits development of new features.
- **Defect Tracking:** Bug reports collected

Bugs forgotten

Examples of issues with ad hoc process

- **Requirements:** Mid-project informal agreement to changes suggested by customer or manager.
- **QA:** Late detection of requirements and design issues.
Test-debug-reimplement cycle limits development of new features.
- **Defect Tracking:** Bug reports collected informally.
- **System Integration:** Integration of independently developed components at the very end of the project.

Examples of issues with ad hoc process

- **Requirements:** Mid-project informal agreement to changes suggested by customer or manager.
- **QA:** Late detection of requirements and design issues.
Test-debug-reimplement cycle limits development of new features.
- **Defect Tracking:** Bug reports collected informally.
- **System Integration:** Integration of components at the very end of the project.

Interfaces out of sync

Examples of issues with ad hoc process

- **Requirements:** Mid-project informal agreement to changes suggested by customer or manager.
- **QA:** Late detection of requirements and design issues.
Test-debug-reimplement cycle limits development of new features.
- **Defect Tracking:** Bug reports collected informally.
- **System Integration:** Integration of independently developed components at the very end of the project.
- **Scheduling:** When project is behind, developers are asked weekly for new estimates.

Examples of issues with ad hoc process

- **Requirements:** Mid-project informal agreement to changes suggested by customer or manager.
- **QA:** Late detection of requirements and design issues. Test-debug-reimplement cycle limits development of new features.
- **Defect Tracking:** Bug reports collected informally.
- **System Integration:** Integration of independently developed components at the very end of the project.
- **Scheduling:** When project is behind developers are asked weekly for new estimates.

Project falls further behind

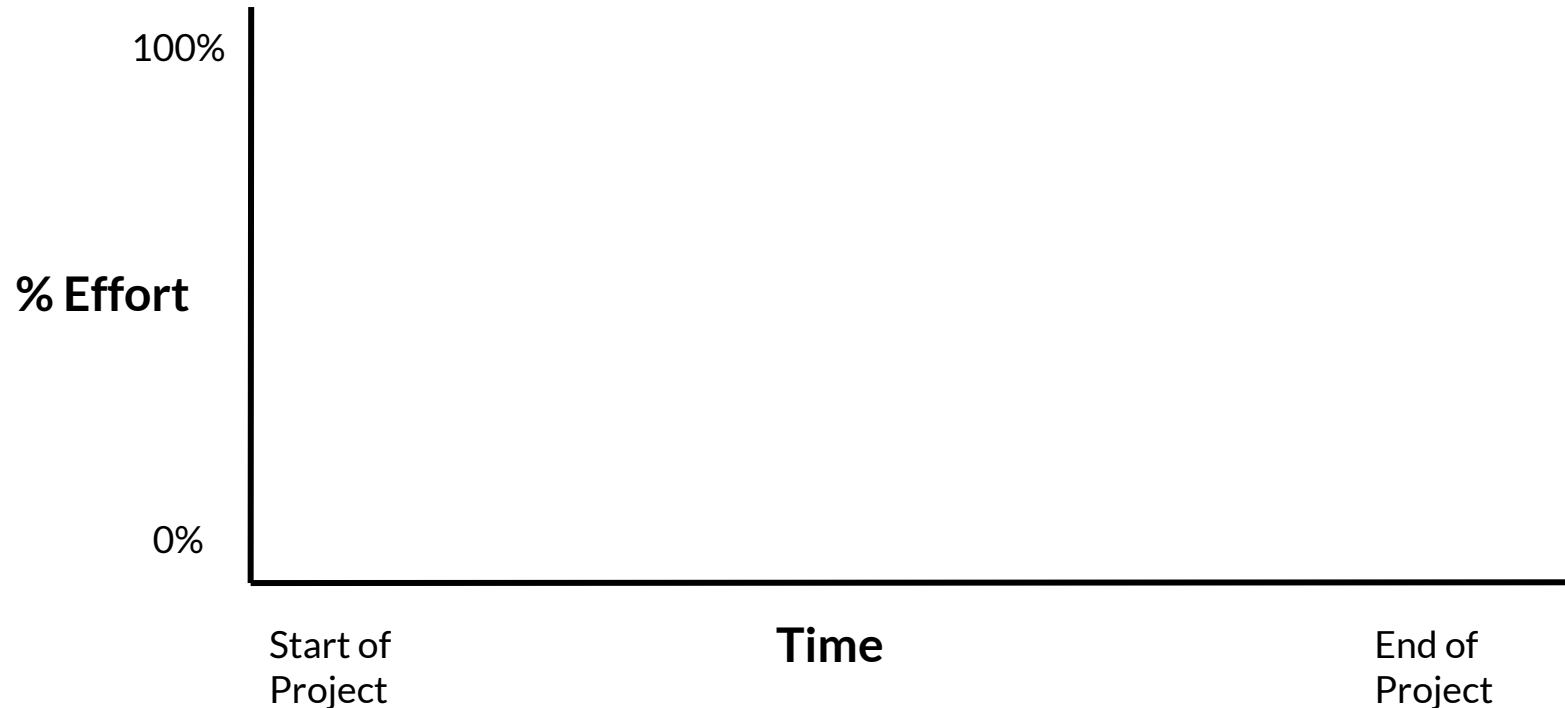
A process hypothesis

- Hypothesis: a good process can **increase flexibility and efficiency** for software development.

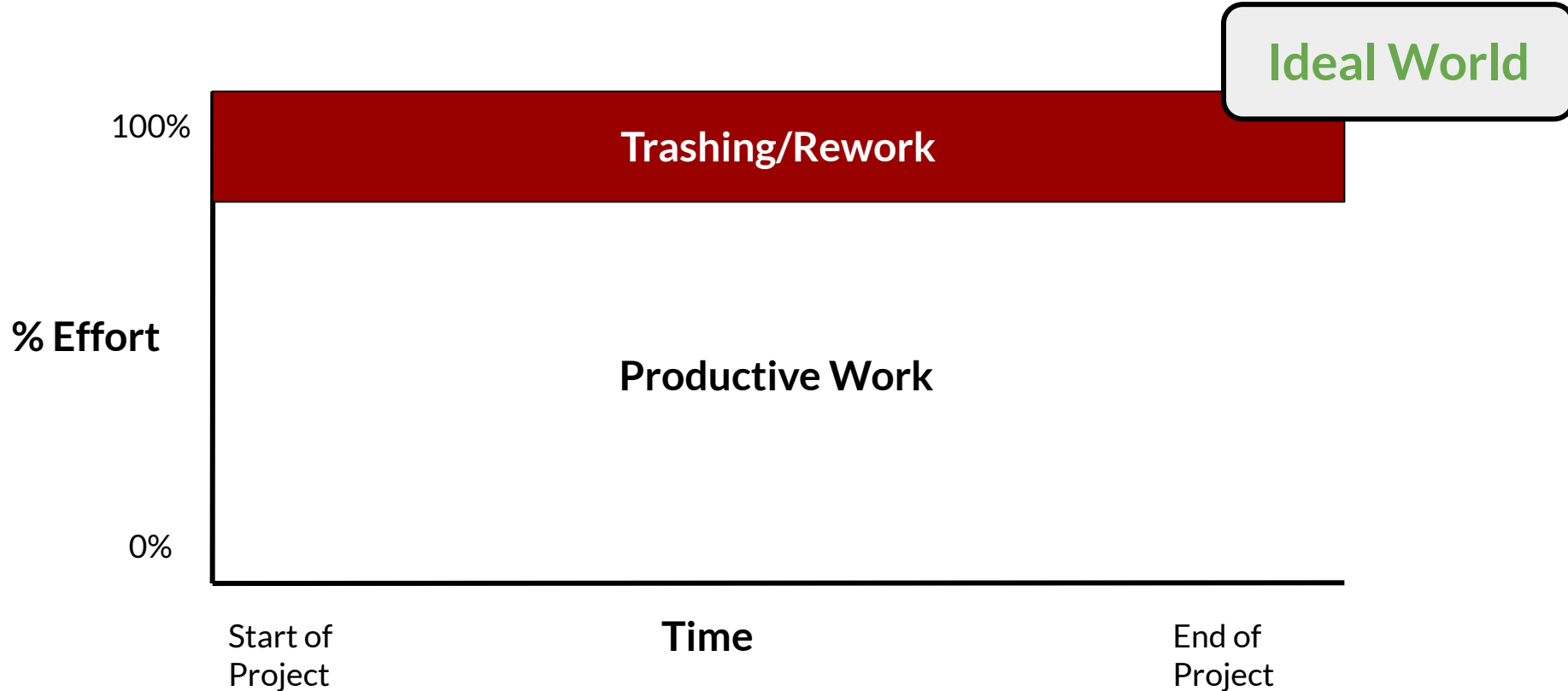
A process hypothesis

- Hypothesis: a good process can **increase flexibility and efficiency** for software development.
 - If this is true, an up-front investment (of resources, e.g., “time”) in process can yield greater returns later on - by avoiding the problems on the previous slide!

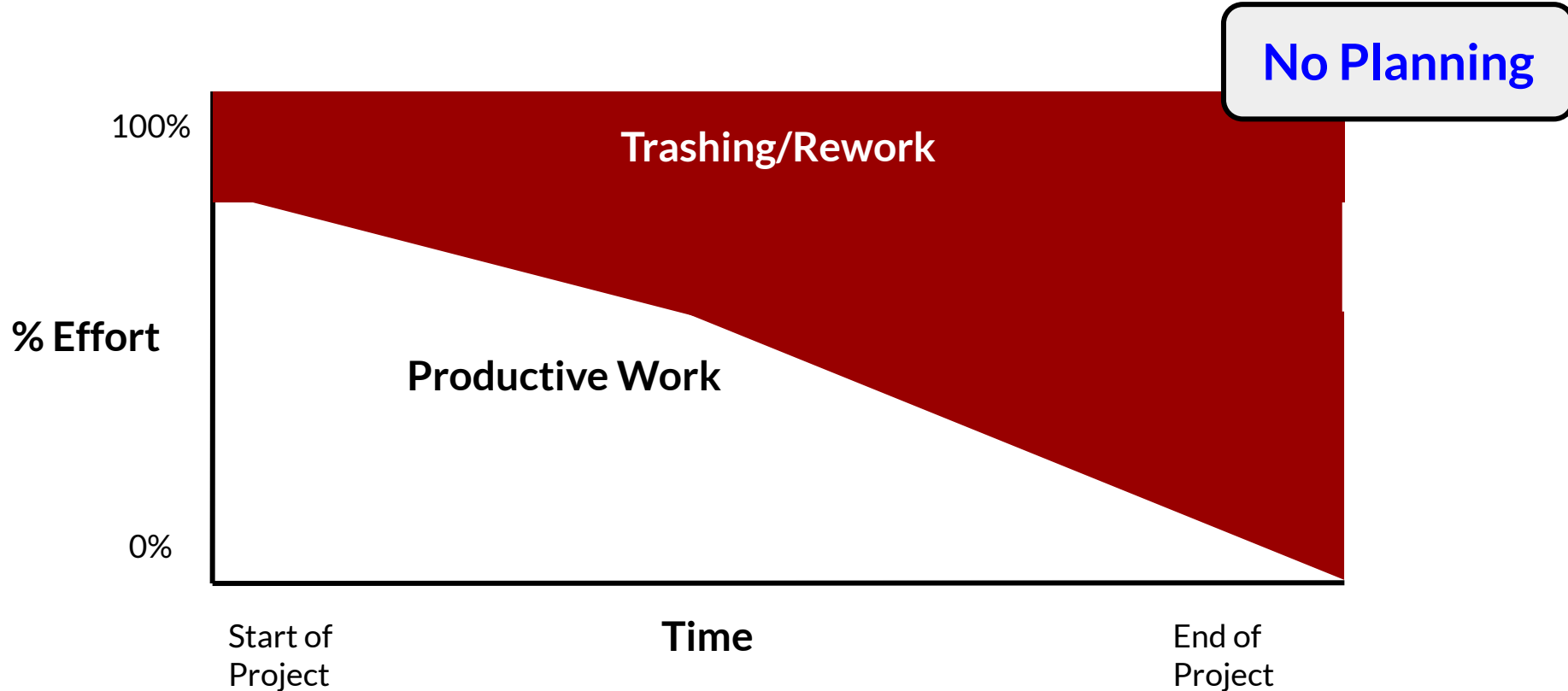
A process hypothesis: activities over time



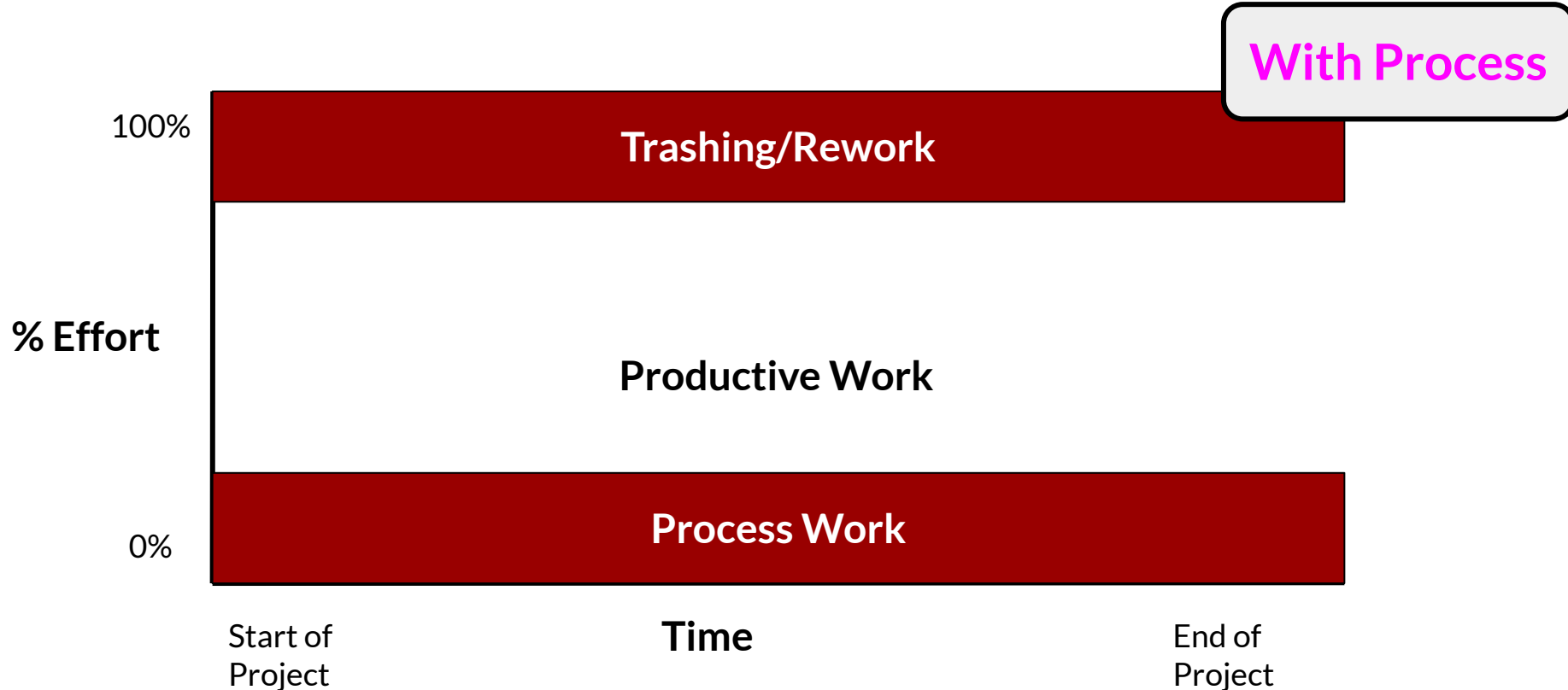
A process hypothesis: activities over time



A process hypothesis: activities over time



A process hypothesis: activities over time



A list of methodologies

- Waterfall
- Spiral
- Agile
- Scrum
- Extreme Programming (XP)
- “wagile”

The Waterfall methodology

In the *waterfall* software development model, the following phases are carried out “**in order**”:

The Waterfall methodology

In the **waterfall** software development model, the following phases are carried out “**in order**”:

- Requirements gathering: produce a document

The Waterfall methodology

In the **waterfall** software development model, the following phases are carried out “**in order**”:

- Requirements gathering: produce a document
- Analysis: create models, schema, and business rules

The Waterfall methodology

In the **waterfall** software development model, the following phases are carried out “**in order**”:

- Requirements gathering: produce a document
- Analysis: create models, schema, and business rules
- Design: create the software architecture

The Waterfall methodology

In the **waterfall** software development model, the following phases are carried out “**in order**”:

- Requirements gathering: produce a document
- Analysis: create models, schema, and business rules
- Design: create the software architecture
- Coding: the development, proving, and integration of software

The Waterfall methodology

In the **waterfall** software development model, the following phases are carried out “**in order**”:

- Requirements gathering: produce a document
- Analysis: create models, schema, and business rules
- Design: create the software architecture
- Coding: the development, proving, and integration of software
- Testing: the systematic discovery and debugging of defects

The Waterfall methodology

In the **waterfall** software development model, the following phases are carried out “**in order**”:

- Requirements gathering: produce a document
- Analysis: create models, schema, and business rules
- Design: create the software architecture
- Coding: the development, proving, and integration of software
- Testing: the systematic discovery and debugging of defects
- Operations: the installation, migration, support, and maintenance of complete systems

The Waterfall methodology

In the **waterfall** software development model, the following phases are carried out “**in order**”:

- Requirements gathering: produce a document
- Analysis: create models, schema, and bus
- Design: create the software architecture
- Coding: the development, proving, and integration of software
- Testing: the systematic discovery and debugging of defects
- Operations: the installation, migration, support, and maintenance of complete systems

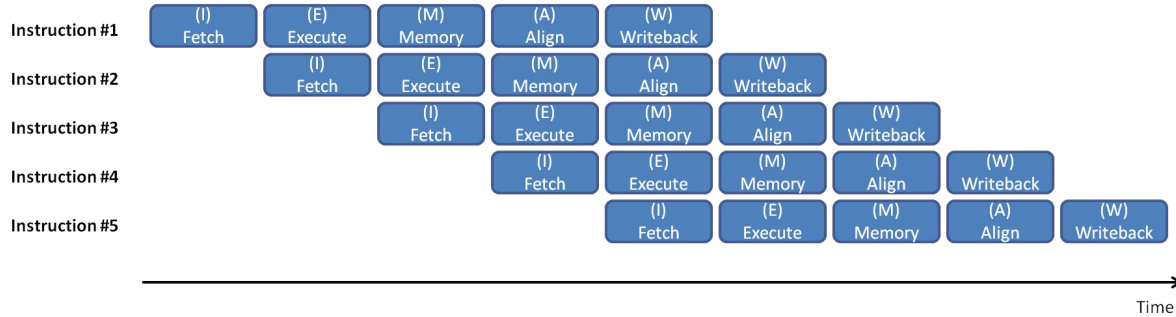
Is this realistic?

Why or why not?

Other lies you've probably been told

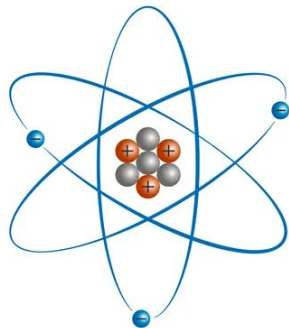
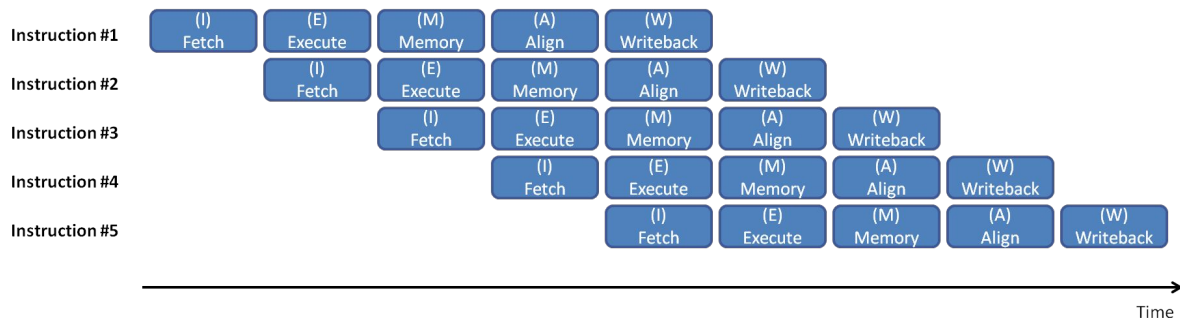
Other lies you've probably been told

PIC32MX Pipelined Instruction Execution



Other lies you've probably been told

PIC32MX Pipelined Instruction Execution

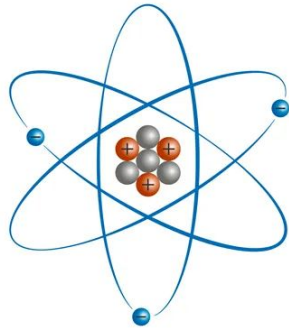
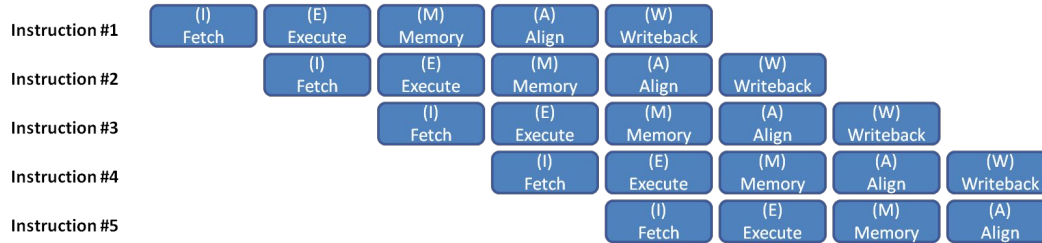


Atom structure

- Proton
- Neutron
- Electron

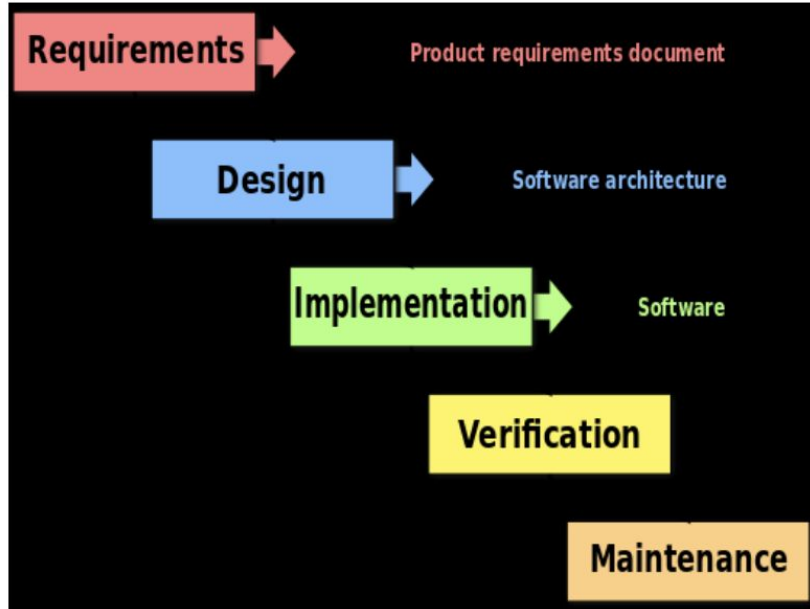
Other lies you've probably been told

PIC32MX Pipelined Instruction Execution



Atom structure

- Proton
- Neutron
- Electron



The Waterfall methodology: an idealized model

- Do **NOT** attempt to actually follow a strict Waterfall methodology in real life
 - you **will** have a bad time

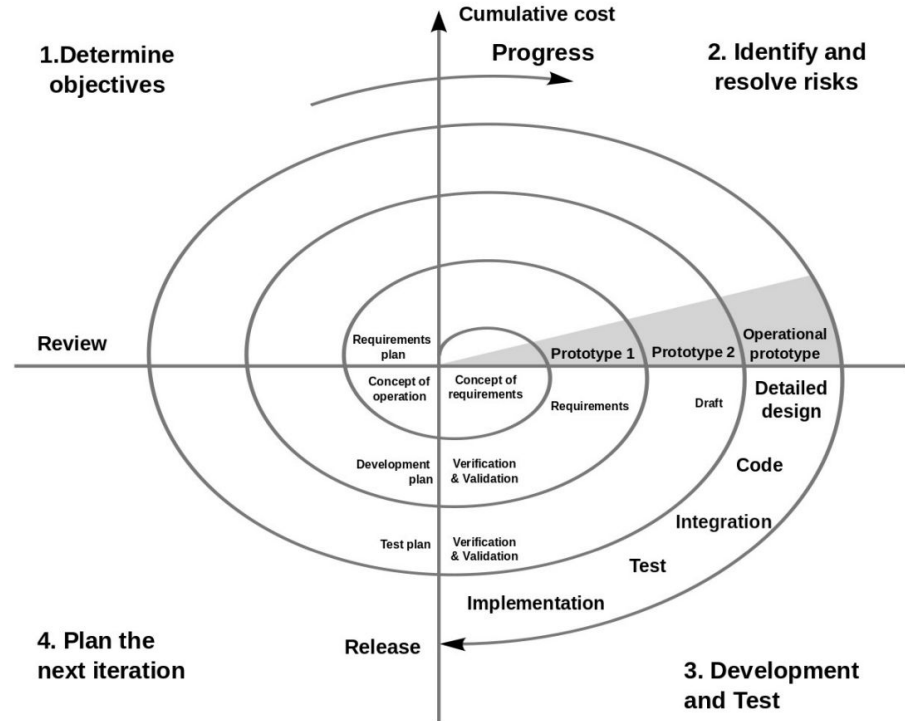
The Waterfall methodology: an idealized model

- Do **NOT** attempt to actually follow a strict Waterfall methodology in real life
 - you **will** have a bad time
- But, it provides a **useful foundation** for thinking about methodologies:
 - the Waterfall stages do represent real activities you'll do during the development lifecycle
 - you probably won't do them all in the proscribed order

The Waterfall methodology: an idealized model

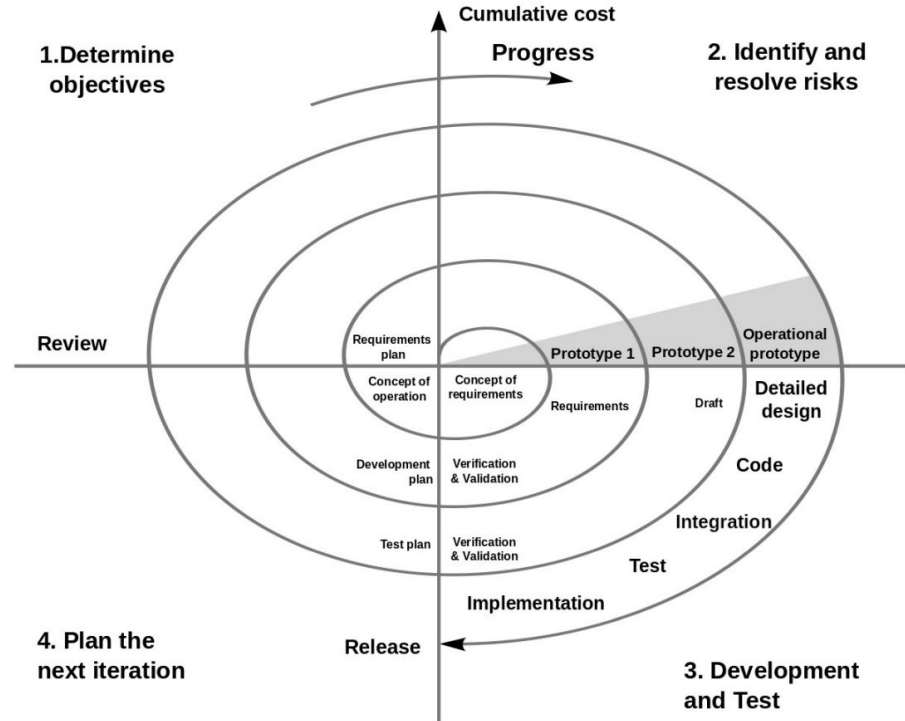
- Do **NOT** attempt to actually follow a strict Waterfall methodology in real life
 - you **will** have a bad time
- But, it provides a **useful foundation** for thinking about methodologies:
 - the Waterfall stages do represent real activities you'll do during the development lifecycle
 - you probably won't do them all in the proscribed order
- And, as we saw in today's reading, this "model" isn't even what the papers "introducing it" were advocating for!

A slightly more realistic model: spiral



A slightly more realistic model: spiral

- Key idea: construct a series of increasingly-complete **prototypes** (while accounting for risk)
- Effectively **iterated waterfall**



A list of methodologies

- Waterfall
- Spiral
- **Agile**
- **Scrum**
- Extreme Programming (XP)
- “wagile”

Agile & Scrum

- Agile is more a **philosophy** than a methodology in the traditional sense

Agile & Scrum

- Agile is more a **philosophy** than a methodology in the traditional sense
- Scrum is an **instantiation** of that philosophy as a methodology

Agile Principles

- Individuals and interactions over processes and tools
- Working software over comprehensive documentation
- Customer collaboration over contract negotiation
- Responding to change over following a plan

Agile Principles

Focus on people

- Individuals and interactions over processes and tools
- Working software over comprehensive documentation
- Customer collaboration over contract negotiation
- Responding to change over following a plan

Agile Principles

- Individuals and interactions over processes and tools
- Working software over comprehensive documentation
- Customer collaboration over contract negotiation
- Responding to change over following a plan

Always have a prototype

Agile Principles

- Individuals and interactions over processes and tools
- Working software over comprehensive documentation
- Customer collaboration over contract negotiation
- Responding to change over following a plan

Keep the client involved

Agile Principles

- Individuals and interactions over processes and tools
- Working software over comprehensive documentation
- Customer collaboration over contract negotiation
- Responding to change over following a plan

Change requirements as you
learn about the problem

The Scrum methodology

- Scrum is one common Agile methodology

The Scrum methodology

- Scrum is one common Agile methodology
- Focused around a “scrum master” who is responsible for process

The Scrum methodology

- Scrum is one common Agile methodology
- Focused around a “scrum master” who is responsible for process
- Work is divided into *sprints* where each team member is responsible for dealing with certain tasks
 - starts with a “sprint planning meeting”: tasks are assigned
 - each day includes a “standup” ceremony
 - at the end of the sprint, a “sprint retrospective meeting” looks back on how the sprint went
 - typically sprints are 1-2 weeks

Common features of Agile methodologies

- Sprint terminology is common, even when not directly using scrum

Common features of Agile methodologies

- Sprint terminology is common, even when not directly using scrum
- “Daily *standups*” are a routine for many engineers

Common features of Agile methodologies

- Sprint terminology is common, even when not directly using scrum
- “Daily *standups*” are a routine for many engineers
- Planning often happens via a *backlog* of *user stories*

Backlogs

- The *product backlog* is a list of all the features for the product that haven't yet been completed

Backlogs

- The **product backlog** is a list of all the features for the product that haven't yet been completed
- The **sprint backlog** is all the features that will be worked on for that sprint. These should be broken down into discrete tasks:
 - Fine-grained
 - Estimated
 - Assigned to individual team members
 - Acceptance criteria should be defined

Backlogs: Kanban



Backlogs

- The **product backlog** is a list of all the features for the product that haven't yet been completed
- The **sprint backlog** is all the features that will be worked on for that sprint. These should be broken down into discrete tasks:
 - Fine-grained
 - Estimated
 - Assigned to individual team members
 - Acceptance criteria should be defined
- **User Stories** are often used to express tasks in the backlog
 - Each user story is a unit of customer-visible functionality

User stories

“As a <role>, I can <capability>, so that I can <receive benefit>.”

User stories

“As a <role>, I can <capability>, so that I can <receive benefit>.”

- also requires a *condition of satisfaction*, which is the measurement you will use to decide if the user story has been completed

User stories

“As a <role>, I can <capability>, so that I can <receive benefit>.”

- also requires a *condition of satisfaction*, which is the measurement you will use to decide if the user story has been completed

You'll need to describe your group project using user stories in your (group) project proposal

User stories: examples

“As a <role>, I can <capability>, so that I can <receive benefit>.”

User stories: examples

“As a <role>, I can <capability>, so that I can <receive benefit>.”

- “As a **computer user**, I want to **backup my entire hard drive** so that **my files are safe**”

User stories: examples

“As a <role>, I can <capability>, so that I can <receive benefit>.”

- “As a **computer user**, I want to **backup my entire hard drive** so that **my files are safe**”
- “As a **typical computer user**, I want to **specify folders to backup**, so that **my most important files are safe**”

User stories: examples

“As a <role>, I can <capability>, so that I can <receive benefit>.”

- “As a **computer user**, I want to **backup my entire hard drive** so that **my files are safe**”
- “As a **typical computer user**, I want to **specify folders to backup**, so that **my most important files are safe**”
- “As a **power user**, I want to **specify subfolders and filetypes NOT to backup**, so that **my backup doesn't fill up with things that I don't need to preserve**”

How do we evaluate a user story?

How do we evaluate a user story?

Remember “**INVEST**”

User stories should be:



Independent

- = “can be scheduled in any order”
- This isn’t always possible, but dependencies should be:
 - minimized
 - clearly labeled when actually necessary



Negotiable

- A good story captures the *essence*, not the details
 - Details can be negotiated during development
- Goal: avoid overly-constraining the implementation
 - e.g., “the light blinks regularly” rather than “the light blinks every two seconds”



Valuable

- This story needs to have value to someone (hopefully the customer!)
 - It's easy to forget why you are doing what you are doing
- Make sure that each story's *benefit* (i.e., 3rd part of the story) clearly expresses its value to the user



Estimable

- Each story should provide enough details to estimate the amount of effort needed to implement it
 - Helps keep the size small
- Best practice: think about how you'd implement the user story as you write it
 - Break down the story if you're not sure how you'd approach it



Small

- Traditional definition: story fits on 3" x 5" index card
- Modern definition: at most two person-weeks of work (i.e., fits into one sprint)
- Too big => unable to estimate



Testable

- Each story must have a clear criterion for when we can mark task “Done”
 - Ensures understanding of task
 - Unable to test == do not understand well enough, need to think more
- Use a measure that can be objectively tested
 - Avoid: “50% of the code is done”
 - OK: “Pressing the button makes *something* happen”
 - (even if you don’t specify what)



Process

Today's agenda:

- Reading Quiz
- Development methodologies
- **Planning, estimation, and risk**
- Measuring progress

Planning

- A project should **plan** time, cost and resources adequately to **estimate** the work needed and to effectively **manage risk** during project execution.

Planning

- A project should **plan** time, cost and resources adequately to **estimate** the work needed and to effectively **manage risk** during project execution.

$$\text{Planning} = \text{estimate} \pm \text{risk}$$

Why is planning a software project difficult?

Why is planning a software project difficult?

- Software tends to be **innovative**

Why is planning a software project difficult?

- Software tends to be **innovative**
 - Cost of copying existing code ≈ 0 , so any project you're actually working on probably is different than what came before

Why is planning a software project difficult?

- Software tends to be **innovative**
 - Cost of copying existing code ≈ 0 , so any project you're actually working on probably is different than what came before
 - "It's not research if you know it's going to work"

Why is planning a software project difficult?

- Software tends to be **innovative**
 - Cost of copying existing code ≈ 0 , so any project you're actually working on probably is different than what came before
 - "It's not research if you know it's going to work"
 - Compare to other kinds of engineering: one highway/bridge/skyscraper/etc isn't that different than the next

Planning: milestones and deliverables

Definition: A *milestone* is a clean end point of a (sub)task

Planning: milestones and deliverables

Definition: A **milestone** is a clean end point of a (sub)task

- Reports, prototypes, completed subprojects, etc.
- “80% done” is **NOT** a suitable milestone (too vague)

Planning: milestones and deliverables

Definition: A *milestone* is a clean end point of a (sub)task

- Reports, prototypes, completed subprojects, etc.
- “80% done” is **NOT** a suitable milestone (too vague)

Definition: A *deliverable* is a milestone that's customer-facing

- “deliverable” is sometimes used interchangeably with “milestone”

Why milestones and deliverables?

- It's easy to tell when a milestone or deliverable is done
- **Progress** towards milestones and deliverables is hard to measure

Why milestones and deliverables?

- It's easy to tell when a milestone or deliverable is done
- **Progress** towards milestones and deliverables is hard to measure

“All I need to do is fix this one bug and then it'll work, promise.”

Estimation

Two parts:

- How long do you think it will take to reach the next milestone?
- Splitting larger tasks into smaller ones

Estimation

Two parts:

- How long do you think it will take to reach the next milestone?
- Splitting larger tasks into smaller ones

Naturally **very fuzzy**: we can't see the future

Estimation techniques: t-shirt sizing



small = I can do this task in an afternoon

Estimation techniques: t-shirt sizing



small = I can do this task in an afternoon



medium = I can do this task in a day or two

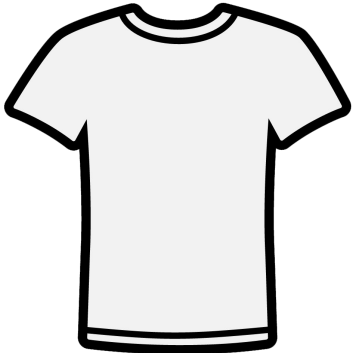
Estimation techniques: t-shirt sizing



small = I can do this task in an afternoon



medium = I can do this task in a day or two



large = too big to estimate how long it will take

- large tasks should usually come with a small task that is breaking the large task up into medium and small tasks

Estimation techniques: story points

- Assign stories 1, 2, 4, or 8 points (these numbers can vary, but the relationship should be **exponential**)
- Like large t-shirt estimates, high-point-value stories should usually have a smaller task to break them apart

Estimation techniques: story points

- Assign stories 1, 2, 4, or 8 points (these numbers can vary, but the relationship should be **exponential**)
- Like large t-shirt estimates, high-point-value stories should usually have a smaller task to break them apart
- T-shirt estimates and story points are two different ways to quantify the **relative** size of tasks
 - Note: “absolute” values of estimates are not very useful
 - Lots of other estimation techniques exist! Use whatever you find most intuitive (and/or, follow your team).

Estimation techniques: cocomo

Definition: a *constructive cost model* (*cocomo*) is a predictive model of time costs based on project history

Estimation techniques: cocomo

Definition: a *constructive cost model* (*cocomo*) is a predictive model of time costs based on project history

- requires experience with similar projects
- rewards documentation of experience
- basically, it's an empirically-derived set of “effort multipliers”.
You multiply the time cost by some numbers from a chart:

Cost Drivers	Ratings					
	Very Low	Low	Nominal	High	Very High	Extra High
Product attributes						
Required software reliability	0.75	0.88	1.00	1.15	1.40	
Size of application database		0.94	1.00	1.08	1.16	
Complexity of the product	0.70	0.85	1.00	1.15	1.30	1.65
Hardware attributes						
Run-time performance constraints			1.00	1.11	1.30	1.66
Memory constraints			1.00	1.06	1.21	1.56
Volatility of the virtual machine environment		0.87	1.00	1.15	1.30	
Required turnabout time		0.87	1.00	1.07	1.15	
Personnel attributes						
Analyst capability	1.46	1.19	1.00	0.86	0.71	
Applications experience	1.29	1.13	1.00	0.91	0.82	
Software engineer capability	1.42	1.17	1.00	0.86	0.70	
Virtual machine experience	1.21	1.10	1.00	0.90		
Programming language experience	1.14	1.07	1.00	0.95		
Project attributes						
Application of software engineering methods	1.24	1.10	1.00	0.91	0.82	
Use of software tools	1.24	1.10	1.00	0.91	0.83	
Required development schedule	1.23	1.08	1.00	1.04	1.10	

Risk and uncertainty

- **Risk management** is the identification, assessment, and prioritization of risks, followed by efforts to minimize, monitor and control unfortunate event outcomes and probabilities.

Risk and uncertainty

- **Risk management** is the identification, assessment, and prioritization of risks, followed by efforts to minimize, monitor and control unfortunate event outcomes and probabilities.
- Any effective plan for software development must take into account common risks, e.g.,:

Risk and uncertainty

- **Risk management** is the identification, assessment, and prioritization of risks, followed by efforts to minimize, monitor and control unfortunate event outcomes and probabilities.
- Any effective plan for software development must take into account common risks, e.g.,:
 - Staff illness or turnover, product is too slow, competitor introduces a similar product, etc.

Risk and uncertainty

- **Risk management** is the identification, assessment, and prioritization of risks, followed by efforts to minimize, monitor and control unfortunate event outcomes and probabilities.
- Any effective plan for software development must take into account common risks, e.g.,:
 - Staff illness or turnover, product is too slow, competitor introduces a similar product, etc.

Bus factor: number of people that need to be simultaneously unavailable until there is a part of the system that no one understands

Strategies for risk management

Strategies for risk management

- Address risk early
- Selectively innovate to increase value while minimizing risk (i.e., focus risk where needed)
- Use iteration and feedback (e.g., prototypes)
- Estimate likelihood and consequences
 - Requires experienced project leads
 - Rough estimates (e.g., <10%, <25%) are OK
- Have contingency plans

Strategies for risk management

- Address risk early
- Selectively innovate to increase value while minimizing risk (i.e., focus risk where needed)
- Use iteration and feedback (e.g., prototypes)
- Estimate likelihood and consequences
 - Requires experienced project lead
 - Rough estimates (e.g., <10%, <25%)
- Have contingency plans

Your ability to do this will come from **practice** (in your first job, senior engineers + tech leads + PMs should do these things)

Process

Today's agenda:

- Finish up VCS slides
- Reading Quiz
- Development methodologies
- Planning, estimation, and risk
- **Measuring progress**

Measuring progress

Easy strategy: only track milestones and deliverables

Measuring progress

Easy strategy: only track milestones and deliverables

- Downside: no way to know how close you are to the next one

Measuring progress

Easy strategy: only track milestones and deliverables

- Downside: no way to know how close you are to the next one

Can we do better? Unfortunately, not really.

Measuring progress: best practices

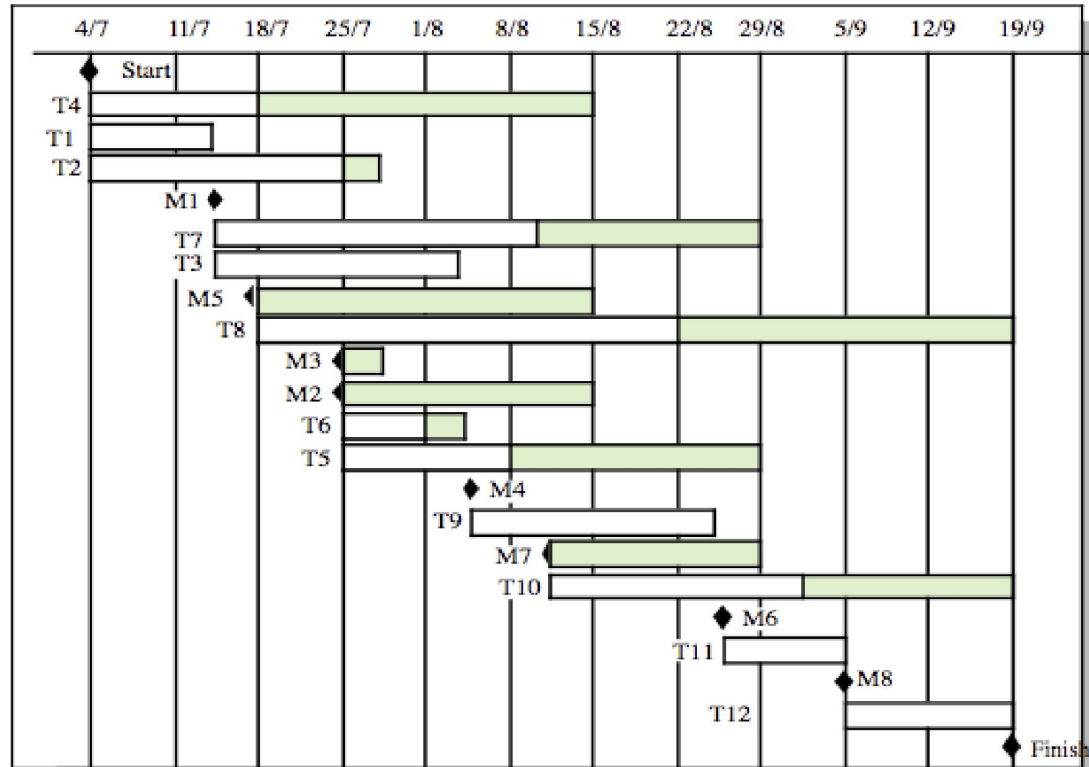
- have many milestones/deliverables
 - think back to Agile: this is a reason to always have a prototype

Measuring progress: best practices

- have many milestones/deliverables
 - think back to Agile: this is a reason to always have a prototype
- avoid relying too heavily on developers' estimates
 - we are bad at estimating
 - “last mile” problem: what seems to be last 10% of the work often takes 40% or more of the development time

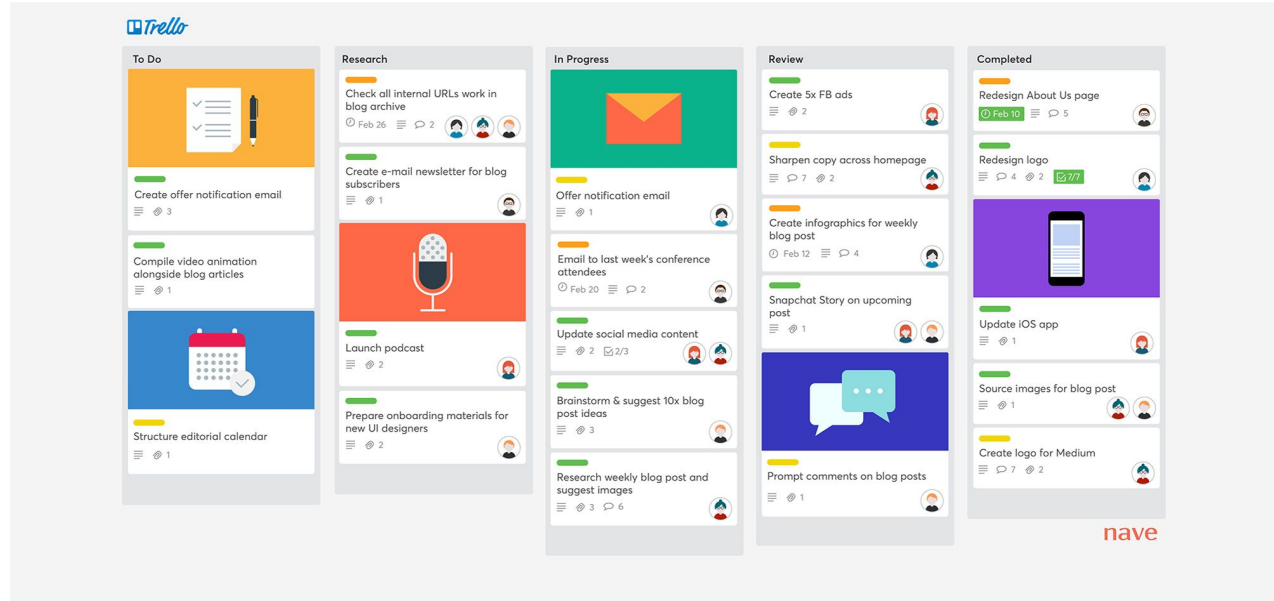
Measuring progress: tools

- Gantt chart



Measuring progress: tools

- Gantt chart
- KanBan board



Measuring progress: tools

- Gantt chart
- KanBan board
- Many others: use what works for you

Takeaways

- Process can save time, but don't overdo it
- Lots of methodologies: choose what makes sense for you
- Agile philosophy is generally a good one to follow
 - But don't focus on it at the expense of actually doing your job
- Estimation is hard and you will get it wrong
 - Use rough estimation strategies to avoid over-promising
- Include lots of buffer + risk in your estimates
- Don't trust developer estimates in general